

Exam Date & Time: 19-Jun-2025 (02:00 PM - 05:00 PM)



MANIPAL ACADEMY OF HIGHER EDUCATION

MIT BLR - BTech VI Semester Makeup Examination, June -2025

PROGRAMMING IN MODERN C PLUS PLUS [NPT-3210]

Marks: 50

Duration: 180 mins.

PART-A

Answer all the questions.

- 1) Analyse the differences between C and C++ by comparing their structure, syntax, and features to find quotient and remainder by writing example program. (4)
 - A)
 - B) You are building a simple data entry module for a system that processes numerical information. This module needs to take user input as a string and determine if it represents a strictly positive integer. Additionally, for valid positive integers, the module should categorize them as either "small" (less than or equal to 100) or "large" (greater than 100). Apply this condition with example program (4)
 - C) Give an example program to illustrate control statements in C++. (2)
- 2) You are mentoring a junior developer who is implementing a stack-based feature for a text editor's undo system. They have written the stack using a C-style struct with global functions (like push, pop, and top). Review their code and suggest converting the implementation to a C++ class with member functions for better object-oriented design. (4)
 - A) Evaluate the benefits and potential drawbacks of using a C++ class with member functions instead of a struct with global functions with stack implementation. (4)
 - B) Create a Rectangle class in C++ that encapsulates the following: (4)

Data:

 - length (private, double)
 - width (private, double)

Methods:

- setDimensions(double l, double w): Sets the length and width of the rectangle (public). Include validation to ensure both length and width are positive values. If invalid values are provided, display an error message and do not update the dimensions.
- calculateArea(): Returns the area of the rectangle (public).
- calculatePerimeter(): Returns the perimeter of the rectangle (public).
- displayRectangleInfo(): Displays the length and width of the rectangle (public).

- C) Describe the role of "this" pointer in C++ and explain with example how it helps identify an object during member function execution. (2)
- 3) You are reviewing the following code which creates two objects obj1 and obj2 of class MyClass. Each object has its own copy of the non-static data member x.
- A)

```
class My Class {  
    int x; // Non-static  
public:  
    void get () { x = 15;}  
    void print () { x = x + 10; cout << "x =" << x << endl; }  
};
```

 (4)
- In main(), both objects call get() to set x to 15, and then print() is used to display the modified value. Evaluate the advantages and limitations of using a non-static data member x in this context. Then, suggest whether using a static data member would improve or hinder the functionality, and justify your reasoning with respect to object-specific behaviour vs shared state.
- B) Designing a system to manage a Report and its corresponding Summary. The Report class contains detailed data, and the Summary class needs to access specific private members of the Report to generate a concise overview. Additionally, you want a standalone utility function print ReportHeader that can also access certain private members of the Report to print a formatted header. Interpret the above scenario by using friend class and friend function. (4)
- C) Explain the difference between pass-by-value and pass-by-reference with respect to how function arguments are handled and potential side effects. (2)
- 4) (4)
- A)

Analyse the output of the provided C++ code, specifically focusing on why the call `pb->f()` results in `"B::f()"` being printed (static binding), while the call `pb->g()` results in `"B::g()"` being printed (dynamic binding).

```
#include <iostream>
using namespace std;
```

```
class B {
public:
    void f() { cout << "B::f()" << endl; }
    virtual void g() { cout << "B::g()" << endl; }
};
```

```
class D : public B {
public:
    void g() { cout << "D::g()" << endl; } // new function
};
```

```
int main() {
    B b;
    D d;

    d.f(); // B::f() -- Inherited
    d.g(); // D::g() -- Added
}
#include <iostream>
using namespace std;
```

```
class B {
public:
    virtual void f() { cout << "B::f()" << endl; }
    virtual void g() { cout << "B::g()" << endl; }
};
```

```
class D : public B {
public:
    virtual void f() { cout << "D::f()" << endl; } //
Overrides B::f()
    virtual void g() { cout << "D::g()" << endl; } //
Overrides B::g()
};
```

```
int main() {
    B b;
    D d;

    b.f(); // B::f()
    b.g(); // B::g()
    d.f(); // D::f() -- Overridden
    d.g(); // D::g() -- Overridden
    // masks the base class function
}
```

B)

(4)

Evaluate the behavior of the non-virtual function `f()`, the virtual function `g()`, and the virtual function `h()` in the provided C++ code when called through a base class pointer `p` that actually points to an object of the most derived class `C`.

```
#include <iostream>
using namespace std;

class A {
public:
    void f() { cout << "A::f()" << endl; }    // Non-
Virtual
    virtual void g() { cout << "A::g()" << endl; } // Virtual
    void h() { cout << "A::h()" << endl; }    // Non-
Virtual
};

class B : public A {
public:
    void f() { cout << "B::f()" << endl; }    // Non-
Virtual
    virtual void g() { cout << "B::g()" << endl; } // Virtual
    virtual void h() { cout << "B::h()" << endl; } // Virtual
};

class C : public B {
public:
    void f() { cout << "C::f()" << endl; }    // Non-
Virtual
    virtual void g() { cout << "C::g()" << endl; } // Virtual
    void h() { cout << "C::h()" << endl; }    // Virtual
};

int main() {
    B *q = new C;
    A *p = q;

    p->f();
    p->g();
    p->h();

    cout << endl;

    q->f();
    q->g();
    q->h();
}
```

```
    cout << endl;

    // Output:
    // A::f()
    // C::g()
    // B::h()
    //
    // B::f()
    // C::g()
    // C::h()

    return 0;
}
```

- C) Describe the primary rule regarding potential data loss during implicit type casting (also known as type promotion) in C (2)
- 5) (4)
- A)

Evaluate the use of rvalue references (&&) and universal references (T&&) in the provided C++ code, specifically focusing on the test, g, f, Data constructor and move constructor, f1, and f2 functions. Explain how std::move and std::forward are employed in conjunction with these reference types to achieve move semantics and perfect forwarding.

```
#include <iostream>
#include <utility>

using namespace std;

// Overloaded functions for test
void test(const int& a) {
    cout << "lvalue ref: " << a << endl;
}

void test(int&& a) {
    cout << "rvalue ref: " << a << endl;
}

template <typename T>
class Data {
public:
    T _data;
    Data(T data) : _data(data) {
        cout << "ctor: " << _data << endl;
    }
    Data(Data&& obj) : _data(move(obj._data)) {
        cout << "mtor: " << _data << endl;
    }
};

void g(int&& param) { // simple rvalue ref
    test(forward<int>(param));
}

template <typename T>
void f(T&& param) { // template fn - universal ref
    test(forward<T>(param));
}

template <typename V>
void f1(V&& v) { // rvalue ref
    test(forward<V>(v));
}
```

```

}

template <typename U>
void f2(U&& v) { // universal ref
    test(forward<U>(v));
}

int main() {
    int a = 20;

    // int&& r = a; // cannot bind rvalue reference of type
    // 'int &&' to lvalue of type int

    g(move(a)); // rvalue: 20
    f(a);       // lvalue: 20
    f(move(a)); // rvalue: 20

    Data<int> d1(10); // ctor: 10
    Data<int> d2(move(d1)); // mtor: 10
    // f1<int&>(d1._data); // cannot bind rvalue reference
    // of type 'int &&' to lvalue of type int
    f1<int>(move(d1._data)); // rvalue: 10
    f2<int&>(d1._data); // lvalue: 10
    f2<int>(move(d1._data)); // rvalue: 10

    return 0;
}

// For std::forward - See Perfect Forwarding

```

B) Analyse and write a program to implement virtual class concept by considering following conditions

1. baseClass:

The baseClass has a virtual function display(). This means that in derived classes, a function with the same name and parameters can override it.

2. derivedClass:

The derivedClass inherits from baseClass and overrides the display() function.

(4)

3. Polymorphism:

In main(), a derivedClass object is created, and its address is stored in a baseClass pointer (ptr).

4. Output:

When ptr->display() is called, the derivedClass version of display() is executed, demonstrating the dynamic binding (runtime polymorphism) achieved through virtual functions.

C) Describe the primary performance benefit of providing "Move support" to classes in C++?

(2)

-----End-----